

第十一章 并发控制

(2026 春季)

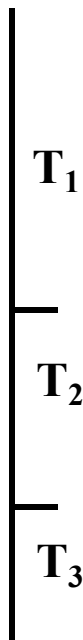


事务并发

- ❑ 多用户数据库系统，允许多个用户同时使用的数据库系统
 - 飞机订票数据库系统
 - 银行数据库系统
- ❑ 特点：在同一时刻并发运行的事务数可达数百上千个
- ❑ 事务并发执行带来的问题
 - 会产生多个事务同时存取同一数据的情况
 - 可能会存取和存储不正确的数据，破坏事务隔离性和数据库的一致性
- ❑ 数据库管理系统必须提供并发控制机制，并发控制机制是衡量一个数据库管理系统性能的重要标志之一

(1) 事务串行执行

- 每个时刻只有一个事务在运行，其他事务必须等到这个事务结束以后方能运行
- 不能充分利用系统资源，发挥数据库共享资源的特点

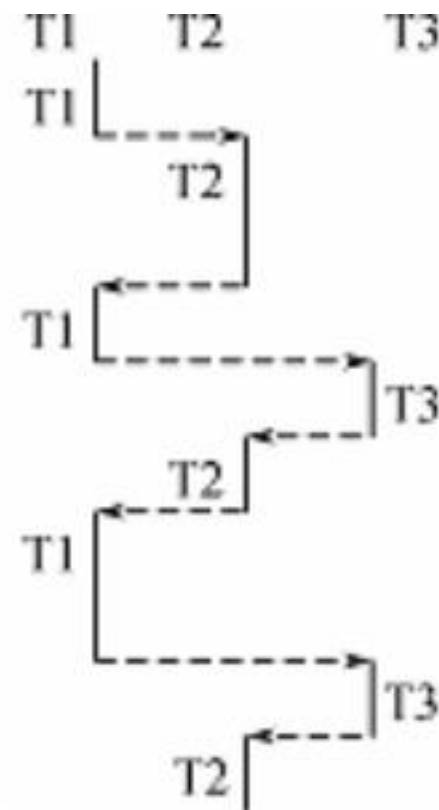


事务的串行执行方式



(2) 交叉并发方式 (Interleaved Concurrency)

- 在单处理机系统中，事务的并行执行是这些并行事务的并行操作轮流交叉运行
- 单处理机系统中的并行事务并没有真正地并行运行，但能够减少处理机的空闲时间，提高系统的效率（充分利用处理机、磁盘等硬件设备之间的并行处理能力）



(b) 事务的交叉并发执行方式

(3) 同时并发方式 (simultaneous concurrency)

- 多处理机系统中，每个处理机可以运行一个事务，多个处理机可以同时运行多个事务，实现多个事务真正的并行运行
- 是最理想的并发方式，但受制于硬件环境
- 需要更复杂的并发控制机制

❑ 在本课程中，只讨论以单处理机系统为基础的事务并发。

并发控制

- ❑ 在本课程中，只讨论以单处理机系统为基础的事务并发。
- ❑ 事务并发执行带来的问题
 - 会产生多个事务同时存取同一数据的情况
 - 可能会存取和存储不正确的数据，破坏事务隔离性和数据库的一致性
- ❑ 数据库管理系统必须提供并发控制机制
- ❑ 并发控制机制是衡量一个数据库管理系统性能的重要标志之一

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

- ❑ 事务是并发控制的基本单位

- ❑ 并发控制机制的任务

- 对并发操作进行正确调度

- 保证事务的隔离性

- 保证数据库的一致性

不一致性的例子

❑ [例11.1]飞机订票系统中的一个活动序列

- ① 甲售票点(事务 T_1)读出某航班的机票余额 A , 设 $A=16$;
- ② 乙售票点(事务 T_2)读出同一航班的机票余额 A , 也为16;
- ③ 甲售票点卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为15, 把 A 写回数据库;
- ④ 乙售票点也卖出一张机票, 修改余额 $A \leftarrow A-1$, 所以 A 为15, 把 A 写回数据库

(结果: 明明卖出两张机票, 数据库中机票余额只减少1)

❑ 这种情况称为数据库的不一致性, 是由并发操作引起的。

- 在并发操作情况下, 对 T_1 、 T_2 两个事务的操作序列的调度是随机的。
- 若按上面的调度序列执行, T_1 事务的修改就被丢失。
 - 原因: 第4步中 T_2 事务修改 A 并写回后覆盖了 T_1 事务的修改

并发操作带来的数据不一致性

❑ 三种数据不一致性

- 丢失修改 (Lost Update)
- 不可重复读 (Non-repeatable Read)
- 读“脏”数据 (Dirty Read)

❑ 记号

- $R(x)$: 读数据 x
- $W(x)$: 写数据 x

丢失修改

- ❑ 两个事务 T_1 和 T_2 读入同一数据并修改， T_2 的提交结果破坏了 T_1 提交的结果，导致 T_1 的修改被丢失。
- ❑ 上面飞机订票例子就属此类

T_1	T_2
① $R(A)=16$	
②	$R(A)=16$
③ $A \leftarrow A-1$	
$W(A)=15$	
④	$A \leftarrow A-1$
	$W(A)=15$

不可重复读 1

- ❑ 不可重复读是指事务 T_1 读取数据后，事务 T_2 执行更新操作，使 T_1 无法再现前一次读取结果。
- ❑ 不可重复读包括三种情况，后两种不可重复读有时也称为**幻影现象**（Phantom Row）或**幻读**（Phantom Read）：
 - (1) 事务 T_1 读取某一数据后，事务 T_2 对其做了修改，当事务 T_1 再次读该数据时，得到与前一次不同的值。
 - (2) 事务 T_1 按一定条件从数据库中读取了某些数据记录后，事务 T_2 删除了其中部分记录，当 T_1 再次按相同条件读取数据时，发现某些记录神秘地消失了。
 - (3) 事务 T_1 按一定条件从数据库中读取某些数据记录后，事务 T_2 插入了一些记录，当 T_1 再次按相同条件读取数据时，发现多了一些记录。

不可重复读 2

T_1	T_2
① $R(A)=50$	
$R(B)=100$	
求和=150	
②	$R(B)=100$
	$B \leftarrow B * 2$
	$W(B)=200$
③ $R(A)=50$	
$R(B)=200$	
求和=250	
(验算不对)	

❑ T_1 读取 $B=100$ 进行运算。

❑ T_2 读取同一数据 B

❑ 对其进行修改

❑ 然后将 $B=200$ 写回数据库。

❑ T_1 为了对读取值校对重读 B ，读取结果 B 的值已为 200 ，与第一次读取到的值不一致！

□ 读“脏”数据是指：

- 事务 T_1 修改某一数据，并将其写回磁盘
- 事务 T_2 读取同一数据后， T_1 由于某种原因被撤销
- 这时 T_1 已修改过的数据恢复原值， T_2 读到的数据就与数据库中的数据不一致
- T_2 读到的数据就为“脏”数据，即不正确的数据

读“脏”数据 2

T_1	T_2
① $R(C)=100$	
$C \leftarrow C * 2$	
$W(C)=200$	
②	$R(C)=200$
③ ROLLBACK	
C恢复为100	

- ❑ T_1 将C值修改为200，并写回数据库
- ❑ T_2 读到C为200（其他并发事务未提交的修改结果）
- ❑ T_1 由于某种原因撤销，其修改作废，C恢复原值100
- ❑ 这时 T_2 持有的C值为200，与数据库内容不一致，就是“脏”数据

数据不一致性及并发控制

- ❑ 数据不一致性：由于并发操作破坏了事务的隔离性
- ❑ 并发控制就是要用正确的方式调度并发操作，使一个用户事务的执行不受其他事务的干扰，从而避免造成数据的不一致性
- ❑ 对数据库的应用有时允许某些不一致性
 - 例如：有些统计工作涉及数据量很大，读到一些“脏”数据对统计精度没什么影响，可以降低对一致性的要求以减少系统开销
 - 一个应用降低对一致性的要求，但不能对其他并发事务的正确运行产生影响（确保事务并发的隔离性）

□ 并发控制的主要技术

- 封锁(Locking)
- 时间戳(Timestamp)
- 乐观控制法
- 多版本并发控制(MVCC)

- 11.1 并发控制概述
- 11.2 封锁
- 11.3 封锁协议
- 11.4 活锁和死锁
- 11.5 并发调度的可串行性
- 11.6 两段锁协议
- 11.7 封锁的粒度
- *11.8 其他并发控制机制
- 11.9 小结

11.2 封锁

- ❑ 什么是封锁
- ❑ 基本封锁类型
- ❑ 锁的相容矩阵

封锁

- ❑ 封锁就是事务T在对某个数据对象（例如表、记录等）操作之前，先向系统发出请求，对其加锁
- ❑ 加锁后事务T就对该数据对象有了一定的控制，在事务T释放它的锁之前，其它的事务不能更新此数据对象。
- ❑ 封锁是实现并发控制的一个非常重要的技术
- ❑ 一个事务对某个数据对象加锁后究竟拥有什么样的控制由封锁的类型决定。
- ❑ 基本封锁类型
 - 排它锁（Exclusive Locks，简记为X锁）
 - 共享锁（Share Locks，简记为S锁）

排它锁与共享锁

❑ ‘排它锁’又称为‘写锁’

- 若事务T对数据对象A加上X锁，则只允许T读取和修改A，其它任何事务都不能再对A加任何类型的锁，直到T释放A上的锁
- 保证其他事务在T释放A上的锁之前不能再读取和修改A

❑ ‘共享锁’又称为‘读锁’

- 若事务T对数据对象A加上S锁，则事务T可以读A但不能修改A，其它事务只能再对A加S锁，而不能加X锁，直到T释放A上的S锁
- 保证其他事务可以读A，但在T释放A上的S锁之前不能对A做任何修改

➤ ‘读锁’ 仅有对被封锁的数据对象的读的权限

➤ ‘写锁’ 不仅有对被封锁的数据对象的写的权限，也有读的权限

锁的相容矩阵

$T_1 \backslash T_2$	X lock	S lock	No lock
X lock	N	N	Y
S lock	N	Y	Y
No lock	Y	Y	Y

其中：

Y=Yes，相容的请求

N=No，不相容的请求

□ 在锁的相容矩阵中：

- 最左边一列表示事务 T_1 已经获得的数据对象上的锁的类型，其中：X lock表示排它锁，S lock表示共享锁，No lock表示没有加锁。
- 最上面一行表示另一事务 T_2 对同一数据对象发出的封锁请求。
- T_2 的封锁请求能否被满足用矩阵中的Y和N表示
 - Y表示事务 T_2 的封锁要求与 T_1 已持有的锁相容，封锁请求可以满足
 - N表示 T_2 的封锁请求与 T_1 已持有的锁冲突， T_2 的请求被拒绝

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

封锁协议

- ❑ 在运用X锁和S锁对数据对象加锁时，需要约定一些规则，这些规则为封锁协议（Locking Protocol）。
 - 何时申请X锁或S锁
 - 持锁时间
 - 何时释放

- ❑ 对封锁方式规定不同的规则，就形成了各种不同的封锁协议，它们分别在不同的程度上为并发操作的正确调度提供一定的保证。
 - 一级封锁协议
 - 二级封锁协议
 - 三级封锁协议

一级封锁协议 1

□ 一级封锁协议

➤ 事务T在修改数据R之前必须先对其加X锁，直到事务结束才释放。

- 正常结束 (COMMIT)
- 非正常结束 (ROLLBACK)

□ 一级封锁协议可防止丢失修改，并保证事务T是可恢复的。

□ 在一级封锁协议中，如果仅仅是读数据不对其进行修改，是不需要加锁的，所以它不能保证可重复读和不读“脏”数据。

一级封锁协议 1

T_1	T_2
① Xlock A	
② $R(A)=16$	
	Xlock A
③ $A \leftarrow A-1$	等待
$W(A)=15$	等待
Commit	等待
Unlock A	等待
④	获得Xlock A
	$R(A)=15$
	$A \leftarrow A-1$
⑤	$W(A)=14$
	Commit
	Unlock A

- ❑ 事务 T_1 在读A进行修改之前，必须先申请对A加X锁并得到批准（加锁成功）
- ❑ 当 T_2 再请求对A加X锁时被拒绝（锁申请请求被阻塞）
- ❑ 只能等待其他所有的事务（ T_1 ）释放A上的锁后， T_2 才能获得对A的X锁
- ❑ T_2 对A加X锁的申请得到批准（加锁成功）
- ❑ 这时， T_2 读到的A已经是 T_1 更新并提交过的值15
- ❑ T_2 按此新的A值进行运算，并将结果值 $A=14$ 写回到数据库磁盘。避免了丢失 T_1 的更新。

二级封锁协议 1

□ 二级封锁协议

➤ 一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，读完后即可释放S锁。

□ 二级封锁协议可以防止丢失修改和读“脏”数据。

□ 在二级封锁协议中，由于读完数据后即可释放S锁，所以它不能保证可重复读。

二级封锁协议 2

T_1	T_2
① Xlock C	
$R(C)=100$	
$C \leftarrow C * 2$	
$W(C)=200$	
②	Slock C
	等待
③ROLLBACK	等待
(C恢复为100)	等待
Unlock C	等待
④	获得Slock C
	$R(C)=100$
⑤	Commit C
	Unlock C

- ❑ 事务 T_1 在对C进行修改之前，先对C加X锁，修改其值后写回磁盘
- ❑ T_2 请求在C上加S锁，因 T_1 已在C上加了X锁， T_2 只能等待
- ❑ T_1 因某种原因被撤销，C恢复为原值100
- ❑ T_1 释放C上的X锁后 T_2 获得C上的S锁，读 $C=100$ 。避免了 T_2 读“脏”数据

□ 三级封锁协议

- 一级封锁协议加上事务T在读取数据R之前必须先对其加S锁，直到事务结束才释放。

□ 三级封锁协议可防止丢失修改、读脏数据和不可重复读。

三级封锁协议 1

T ₁	T ₂
① Slock A	
Slock B	
R(A)=50	
R(B)=100	
求和=150	
②	Xlock B
	等待
③ R(A)=50	等待
R(B)=100	等待
求和=150	等待
Commit	等待
Unlock A	等待
Unlock B	等待
④	获得XlockB
	R(B)=100
	B←B*2
⑤	W(B)=200
	Commit
	Unlock B

- ❑ 事务T₁在读A，B之前，先对A和B加S锁。
- ❑ 之后，其他事务只能再对A和B加S锁，而不能加X锁，即其他事务只能读A和B，而不能修改。
- ❑ 当T₂为修改B而申请对B的X锁时被拒绝，只能等待T₁释放B上的锁。
- ❑ T₁为验算再读A和B，这时读出的B仍是100，求和结果仍为150，满足“可重复读”。
- ❑ T₁运行结束，释放T₁所持有的A和B上的S锁。
- ❑ 此时，T₂才能获得对B的X锁

封锁协议小结

❑ 三级协议的主要区别

➤ 什么操作需要申请封锁以及何时释放锁（即持锁时间）

❑ 不同的封锁协议使事务达到的一致性级别不同

➤ 封锁协议级别越高，一致性程度越高

	X锁		S锁		一致性保证		
	操作结束释放	事务结束释放	操作结束释放	事务结束释放	不丢失修改	不读“脏”数据	可重复读
一级封锁协议		✓			✓		
二级封锁协议		✓	✓		✓	✓	
三级封锁协议		✓		✓	✓	✓	✓

表11.1 不同级别的封锁协议和一致性保证

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

11.4 活锁和死锁

❑ 封锁技术可以有效地解决并行操作的一致性问题，但也带来一些新的问题

➤ 死锁

➤ 活锁

活锁 1

❑ 假设在并发控制的实现中，只采用一种类型的封锁‘排它锁’。

- ① 事务 T_1 封锁了数据 R 。
- ② 事务 T_2 又请求封锁 R ，于是 T_2 等待。
- ③ T_3 也请求封锁 R ，当 T_1 释放了 R 上的封锁之后系统首先批准了 T_3 的请求， T_2 仍然等待。
- ④ T_4 又请求封锁 R ，当 T_3 释放了 R 上的封锁之后系统又批准了 T_4 的请求.....
- ⑤ T_2 有可能永远等待，这就是活锁的情形。

❑ 避免活锁：采用先来先服务的策略

- 当多个事务请求封锁同一数据对象时
- 按请求封锁的先后次序对这些事务排队（申请/等待队列）
- 该数据对象上的锁一旦释放，首先批准申请队列中的第一个事务获得锁

活锁 2

T ₁	T ₂	T ₃	T ₄
Lock R	• • •	• • •	• • •
•	Lock R		
•	等待	Lock R	
•	等待	等待	Lock R
Unlock R	等待	等待	等待
	等待	Lock R	等待
•	等待	•	等待
•	等待	Unlock	等待
•	等待	•	Lock R
	等待	•	• •

图11.5 (a) 活锁

死锁

- ① 事务 T_1 封锁了数据 R_1
- ② T_2 封锁了数据 R_2
- ③ T_1 又请求封锁 R_2 ，因 T_2 已封锁了 R_2 ，于是 T_1 等待 T_2 释放 R_2 上的锁
- ④ 接着 T_2 又申请封锁 R_1 ，因 T_1 已封锁了 R_1 ， T_2 也只能等待 T_1 释放 R_1 上的锁
- ⑤ 这样 T_1 在等待 T_2 ，而 T_2 又在等待 T_1 ， T_1 和 T_2 两个事务永远不能结束，形成死锁

□ 解决死锁的方法（两类方法）

1. 死锁的预防

2. 死锁的诊断与解除

T_1	T_2
Lock R_1	•
	•
	•
•	Lock R_2
•	•
•	•
Lock R_2	•
等待	
等待	
等待	Lock R_1
等待	等待
等待	等待
	•
	•
	•

图11.5 (b) 死锁

死锁的预防

- ❑ 产生死锁的原因是两个或多个事务都已封锁了一些数据对象，然后又都请求对已为其他事务封锁的数据对象加锁，从而出现死等待。
- ❑ 预防死锁的发生就是要破坏产生死锁的条件。

- ❑ **一次封锁法**：要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行。

- ❑ 存在的问题

- 降低系统并发度
- 难于事先精确确定封锁对象
 - 数据库中数据是不断变化的，原来不要求封锁的数据，在执行过程中可能会变成封锁对象，所以很难事先精确地确定每个事务所要封锁的数据对象。
 - 解决方法：将事务在执行过程中可能要封锁的数据对象全部加锁，这就进一步降低了并发度。

- ❑ **顺序封锁法**：预先对数据对象规定一个封锁顺序，所有事务都按这个顺序实行封锁。

- ❑ 存在的问题。

- 维护成本

- 数据库系统中封锁的数据对象极多，并且随数据的插入、删除等操作而不断地变化，要维护这样的资源的封锁顺序非常困难，成本很高。

- 难以实现

- 事务的封锁请求可以随着事务的执行而动态地决定，很难事先确定每一个事务要封锁哪些对象，因此也就很难按规定的顺序去施加封锁

死锁的诊断

- ❑ 数据库管理系统在解决死锁的问题上更普遍采用的是诊断并解除死锁的方法
- ❑ 在操作系统中广为采用的预防死锁的策略并不太适合数据库的特点

- ❑ **超时法**：如果一个事务的等待时间超过了规定的时限，就认为发生了死锁。

- ❑ 优点

- 实现简单

- ❑ 缺点

- 有可能误判死锁
 - 时限若设置得太长，死锁发生后不能及时发现

- ❑ **等待图法**：并发控制子系统周期性地（比如每隔数秒）生成事务等待图，检测事务。如果发现图中存在回路，则表示系统中出现了死锁。

- ❑ 事务等待图

- 事务等待图是一个有向图 $G=(T, U)$
 - T 为结点的集合，每个结点表示正运行的事务
 - U 为边的集合，每条边表示事务等待的情况
 - 若 T_1 等待 T_2 ，则在 T_1 与 T_2 之间划一条有向边，从 T_1 指向 T_2

等待图法

- ❑ 图(a)中，事务 T_1 等待 T_2 ， T_2 等待 T_1 ，产生了死锁
- ❑ 图(b)中，事务 T_1 等待 T_2 ， T_2 等待 T_3 ， T_3 等待 T_4 ， T_4 又等待 T_1 ，产生了死锁
- ❑ 图(b)中，事务 T_3 可能还等待 T_2 ，在大回路中又有小的回路

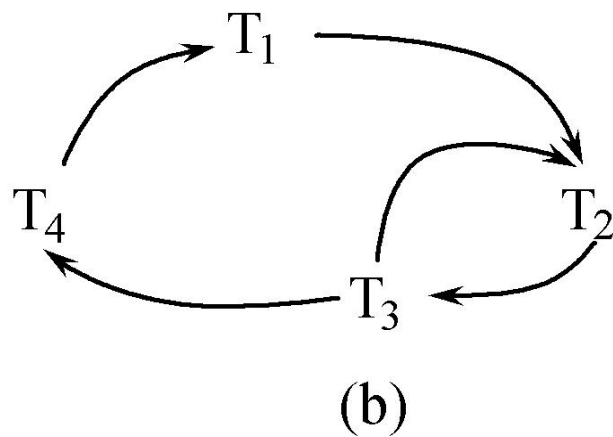
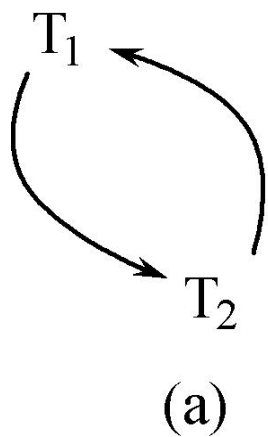


图11.6 事务等待图

□ 解除死锁

- 选择一个处理死锁代价最小的事务，将其撤消
- 释放此事务持有的所有的锁，使其它事务能继续运行下去

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

并发调度的可串行性

- ❑ 数据库管理系统对并发事务不同的调度可能会产生不同的结果
- ❑ ‘串行调度’是正确的
- ❑ 执行结果等价于串行调度的调度也是正确的，称为‘可串行化调度’

❑ 11.5.1 可串行化调度

❑ 11.5.2 冲突可串行化调度



□ 可串行化(Serializable)调度

- 多个事务的并发执行是正确的，当且仅当其结果与按某一次序串行地执行这些事务时的结果相同

□ 可串行性(Serializability)

- 是并发事务正确调度的准则
- 一个给定的并发调度，当且仅当它是可串行化的，才认为是正确调度

可串行化调度的例子

□ [例11.2]现在有两个事务，分别包含下列操作（**A**和**B**是数据库中的两个不同的数据项）

➤ 事务 T_1 ：读**B**； $A=B+1$ ； 写回**A**

➤ 事务 T_2 ：读**A**； $B=A+1$ ； 写回**B**

➤ 现给出对这两个事务不同的调度策略

串行调度, 正确的调度

T ₁	T ₂
Slock B	
Y=R(B)=2	
Unlock B	
Xlock A	
A=Y+1=3	
W(A)	
Unlock A	
	Slock A
	X=R(A)=3
	Unlock A
	Xlock B
	B=X+1=4
	W(B)
	Unlock B

串行调度 (a)

- ❑ 假设A、B的初值均为2。
- ❑ 按T₁→T₂次序执行，结果为 A=3，B=4
- ❑ 串行调度策略，正确的调度

串行调度,正确的调度

T_1	T_2
	Slock A
	$X=R(A)=2$
	Unlock A
	Xlock B
	$B=X+1=3$
	W(B)
	Unlock B
Slock B	
$Y=R(B)=3$	
Unlock B	
Xlock A	
$A=Y+1=4$	
W(A)	
Unlock A	

串行调度 (b)

- ❑ 假设A、B的初值均为2。
- ❑ $T_2 \rightarrow T_1$ 次序执行，结果为B=3，A=4
- ❑ 串行调度策略，正确的调度

不可串行化调度，错误的调度

T ₁	T ₂
Slock B	
Y=R(B)=2	
	Slock A
	X=R(A)=2
Unlock B	
	Unlock A
Xlock A	
A=Y+1=3	
W(A)	
	Xlock B
	B=X+1=3
	W(B)
Unlock A	
	Unlock B

- ❑ 执行结果与调度(a)、调度(b)的结果都不同
- ❑ 是错误的调度

(c) 不可串行化的调度

可串行化调度，正确的调度

T ₁	T ₂
Slock B	
Y=R(B)=2	
Unlock B	
Xlock A	
	Slock A
A=Y+1=3	等待
W(A)	等待
Unlock A	等待
	X=R(A)=3
	Unlock A
	Xlock B
	B=X+1=4
	W(B)
	Unlock B

❑ 执行结果与串行调度(a)的执行结果相同

❑ 是正确的调度

(d) 可串行化的调度

□ 11.5.1 可串行化调度

□ 11.5.2 冲突可串行化调度

□ 冲突可串行化

- 一个比可串行化更严格的条件
- 商用系统中的调度器采用

□ 冲突操作：是指不同的事务对同一数据的读写操作和写写操作：

- $R_i(x)$ 与 $W_j(x)$ /*事务 T_i 读 x , T_j 写 x , 其中 $i \neq j$ */
- $W_i(x)$ 与 $W_j(x)$ /*事务 T_i 写 x , T_j 写 x , 其中 $i \neq j$ */

□ 来自不同事务的其他操作是不冲突操作

- 访问不同的数据对象，或者
- 两者都是读操作

□ 不能交换 (Swap) 的动作：

- 同一事务的两个操作
- 不同事务的冲突操作

冲突可串行化 2

- 一个调度 S_c 在保证冲突操作的次序不变的情况下，通过若干次“**交换相邻‘不冲突操作’的次序**”得到另一个调度 S_c' ，如果 S_c' 是串行的，称调度 S_c 是‘**冲突可串行化**’的调度。
- 若一个调度是冲突可串行化，则一定是可串行化的调度
- 可用这种方法判断一个调度是否是冲突可串行化的

冲突可串行化 3

□ [例11.3] 今有调度 Sc_1 ，通过若干次不冲突操作的次序交换得到调度 Sc_2

Sc_1 $r_1(A); w_1(A); r_2(A); w_2(A); r_1(B); w_1(B); r_2(B); w_2(B);$

$r_1(A); w_1(A); r_2(A); r_1(B); w_2(A); w_1(B); r_2(B); w_2(B);$

$r_1(A); w_1(A); r_1(B); r_2(A); w_2(A); w_1(B); r_2(B); w_2(B);$

$r_1(A); w_1(A); r_1(B); r_2(A); w_1(B); w_2(A); r_2(B); w_2(B);$

Sc_2 $r_1(A); w_1(A); r_1(B); w_1(B); r_2(A); w_2(A); r_2(B); w_2(B);$

□ 因为 Sc_2 是一个串行调度，所以 Sc_1 是一个冲突可串行化的调度

冲突可串行化调度

- ❑ 冲突可串行化调度是可串行化调度的充分条件，不是必要条件。
- ❑ 存在不满足冲突可串行化条件的可串行化调度。
- ❑ [例11.4]有3个事务， $T_1=W_1(Y);W_1(X)$ ， $T_2=W_2(Y);W_2(X)$ ， $T_3=W_3(X)$
 - 调度 $L_1 = W_1(Y);W_1(X);W_2(Y);W_2(X);W_3(X)$ 是一个串行调度
 - 调度 $L_2 = W_1(Y);W_2(Y);W_2(X);W_1(X);W_3(X)$ 不满足冲突可串行化
 - 但是调度 L_2 是可串行化的，因为 L_2 执行的结果与调度 L_1 相同， Y 的值都等于 T_2 的值， X 的值都等于 T_3 的值

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

两段锁协议 1

- ❑ 数据库管理系统普遍采用两段锁协议的方法实现并发调度的可串行性，从而保证调度的正确性
- ❑ 两段锁协议，指所有事务必须分两个阶段对数据项加锁和解锁
 - 在对任何数据进行读、写操作之前，事务首先要获得对该数据的封锁
 - 在释放一个封锁之后，事务不再申请和获得任何其他封锁

两段锁协议 2

□ “两段”锁的含义，事务分为两个阶段

- 第一阶段是获得封锁，也称为**扩展阶段**
 - 事务可以申请获得任何数据项上的任何类型的锁，但是不能释放任何锁
- 第二阶段是释放封锁，也称为**收缩阶段**
 - 事务可以释放任何数据项上的任何类型的锁，但是不能再申请任何锁

□ 例：

- 事务 T_i 遵守两段锁协议，其封锁序列是：

Slock A Slock B Xlock C Unlock B Unlock A Unlock C;

|← 扩展阶段 →| |← 收缩阶段 →|

- 事务 T_j 不遵守两段锁协议，其封锁序列是：

Slock A Unlock A Slock B Xlock C Unlock C Unlock B;

两段锁协议 3

事务T ₁	事务T ₂
Slock A R(A)=260	
	Slock C R(C)=300
Xlock A W(A)=160	
	Xlock C W(C)=250
	Slock A
Slock B R(B)=1000	等待
Xlock B W(B)=1100	等待
Unlock A	等待
	等待
	R(A)=160
	Xlock A
Unlock B	
	W(A)=210
	Unlock C

❑ 左图的调度是遵守两段锁协议的，因此一定是一个可串行化调度。

❑ 如何验证？

遵守两段锁协议的可串行化调度

两段锁协议 4

- ❑ 事务遵守两段锁协议是可串行化调度的充分条件，而不是必要条件。
- ❑ 若并发事务都遵守两段锁协议，则对这些事务的任何并发调度策略都是可串行化的
- ❑ 若并发事务的一个调度是可串行化的，不一定所有事务都符合两段锁协议

❑ 两段锁协议与防止死锁的一次封锁法

- 一次封锁法要求每个事务必须一次将所有要使用的数据全部加锁，否则就不能继续执行，因此一次封锁法遵守两段锁协议
- 但是两段锁协议并不要求事务必须一次将所有要使用的数据全部加锁，因此遵守两段锁协议的事务可能发生死锁

❑ [例]遵守两段锁协议的事务发生死锁

事务 T_1	事务 T_2
Slock B	
R(B)=2	
	Slock A
	R(A)=2
Xlock A	
等待	Xlock A
等待	等待

遵守两段锁协议的事务可能发生死锁

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

封锁粒度

- ❑ 封锁对象的大小称为封锁粒度(Granularity)
- ❑ 封锁的对象：逻辑单元，物理单元
- ❑ 例：在关系数据库中，封锁对象：
 - 逻辑单元：属性值、属性值的集合、元组、关系、索引项、整个索引、整个数据库等
 - 物理单元：页（数据页或索引页）、物理记录等

选择封锁粒度原则

❑ 封锁粒度与系统的并发度和并发控制的开销密切相关。

- 封锁的粒度越大，数据库所能够封锁的数据单元就越少，并发度就越小，系统开销也越小；
- 封锁的粒度越小，并发度较高，但系统开销也就越大

❑ 例

- 若封锁粒度是数据页，事务 T_1 需要修改元组 L_1 ，则 T_1 必须对包含 L_1 的整个数据页 A 加锁。如果 T_1 对 A 加锁后事务 T_2 要修改 A 中元组 L_2 ，则 T_2 被迫等待，直到 T_1 释放 A 。
- 如果封锁粒度是元组，则 T_1 和 T_2 可以同时 L_1 和 L_2 加锁，不需要互相等待，提高了系统的并行度。
- 又如，事务 T 需要读取整个表，若封锁粒度是元组， T 必须对表中的每一个元组加锁，开销极大

❑ 多粒度封锁(Multiple Granularity Locking)

➤ 在一个系统中同时支持多种封锁粒度供不同的事务选择

❑ 选择封锁粒度，同时考虑封锁开销和并发度两个因素，适当选择封锁粒度

➤ 需要处理多个关系的大量元组的用户事务：以数据库为封锁单位

➤ 需要处理大量元组的用户事务：以关系为封锁单元

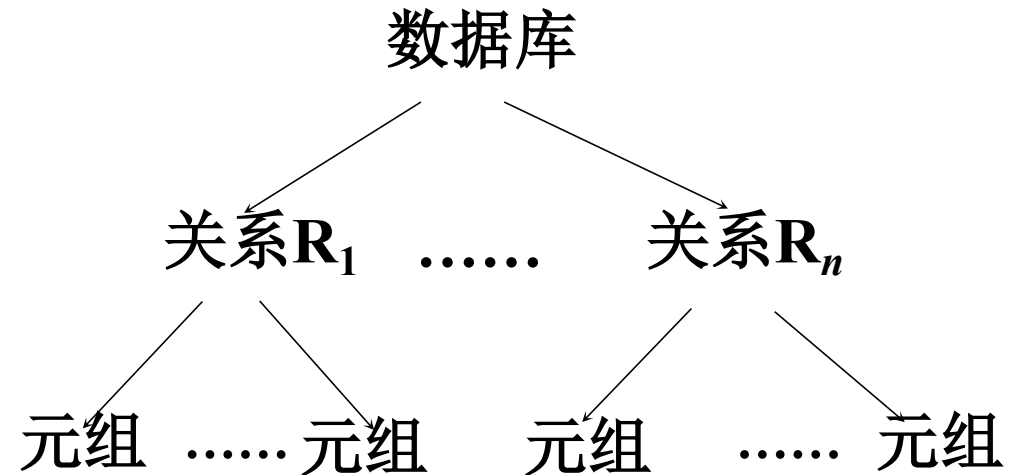
➤ 只处理少量元组的用户事务：以元组为封锁单位

❑ 多粒度树

- 以树形结构来表示多级封锁粒度
- 根节点是整个数据库，表示最大的数据粒度
- 叶结点表示最小的数据粒度

❑ 例：三级粒度树

- 根结点为数据库
- 数据库的子结点为关系
- 关系的子结点为元组。



多粒度封锁协议

- ❑ 允许多粒度树中的每个结点被独立地加锁
- ❑ 对一个结点加锁意味着这个结点的所有后裔结点也被加以同样类型的锁
- ❑ 在多粒度封锁中一个数据对象可能以两种方式封锁：
 - **显式封锁**：接加到数据对象上的封锁
 - **隐式封锁**：是该数据对象没有独立加锁，是由于其上级结点加锁而使该数据对象加上了锁
- ❑ 显式封锁和隐式封锁的效果是一样的

❑ 系统检查封锁冲突时

- 要检查显式封锁
- 还要检查隐式封锁

❑ 例如，事务T要对关系 R_1 加X锁

- 系统必须搜索其上级结点 数据库、关系 R_1
- 还要搜索 R_1 的下级结点，即 R_1 中的每一个元组
- 如果其中某一个数据对象已经加了不相容锁，则T必须等待

❑ 对某个数据对象加锁，系统要检查

➤ 该数据对象

- 有无显式封锁与之冲突

➤ 所有上级结点

- 检查本事务的显式封锁是否与该数据对象上的隐式封锁冲突：
(由上级结点已加的封锁造成的)

➤ 所有下级结点

- 看上面的显式封锁是否与本事务的隐式封锁（将加到下级结点的封锁）冲突

意向锁

❑ 引进意向锁 (intention lock) 目的

➤ 提高对某个数据对象加锁时系统的检查效率

❑ 如果对一个结点加意向锁，则说明该结点的下层结点正在被加锁

❑ 对任一结点加基本锁，必须先对它的上层结点加意向锁

➤ 例如，对任一元组加锁时，必须先对它所在的数据库和关系加意向锁

常用意向锁

□ 意向共享锁(Intent Share Lock, 简称IS锁)

- 如果对一个数据对象加IS锁, 表示它的后裔结点拟(意向)加S锁。
- 例如: 事务 T_1 要对 R_1 中某个元组加S锁, 则要首先对关系 R_1 和数据库加IS锁

□ 意向排它锁(Intent Exclusive Lock, 简称IX锁)

- 如果对一个数据对象加IX锁, 表示它的后裔结点拟(意向)加X锁。
- 例如: 事务 T_1 要对 R_1 中某个元组加X锁, 则要首先对关系 R_1 和数据库加IX锁

□ 共享意向排它锁(Share Intent Exclusive Lock, 简称SIX锁)

- 如果对一个数据对象加SIX锁, 表示对它加S锁, 再加IX锁, 即 $SIX = S + IX$ 。
- 例: 对某个表加SIX锁, 则表示该事务要读整个表(所以要对该表加S锁), 同时会更新个别元组(所以要对该表加IX锁)。

意向锁的相容矩阵

$T_1 \backslash T_2$	S	X	IS	IX	SIX	-
S	Y	N	Y	N	N	Y
X	N	N	N	N	N	Y
IS	Y	N	Y	Y	Y	Y
IX	N	N	Y	Y	N	Y
SIX	N	N	Y	N	N	Y
-	Y	Y	Y	Y	Y	Y

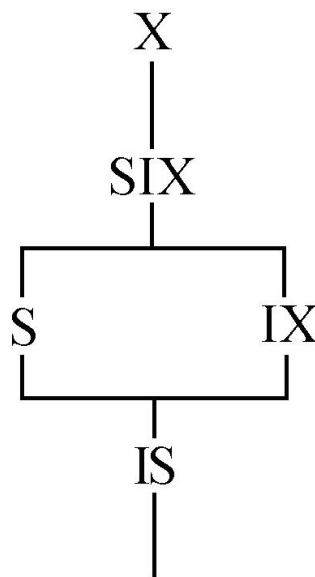
Y=Yes, 表示相容的请求

N=No, 表示不相容的请求

(a) 数据锁的相容矩阵

锁的强度

- ❑ 锁的强度是指它对其他锁的排斥程度
- ❑ 一个事务在申请封锁时以强锁代替弱锁是安全的，反之则不然



(b) 锁的强度的偏序关系

具有意向锁的多粒度封锁方法

❑ 申请封锁时应该按自上而下的次序进行

❑ 释放封锁时则应该按自下而上的次序进行

❑ 具有意向锁的多粒度封锁方法

- 提高了系统的并发度
- 减少了加锁和解锁的开销
- 在实际的数据库管理系统产品中得到广泛应用

❑ 例如：事务 T_1 要对关系 R_1 加S锁

- 要首先对数据库加IS锁（检查数据库是否已加了不相容的X锁）
- 申请对关系 R_1 加S锁（检查关系 R_1 是否已加了不相容的X、IX或SIX锁）
- 不再需要搜索和检查 R_1 中的元组是否加了不相容的锁。

- ❑ 11.1 并发控制概述
- ❑ 11.2 封锁
- ❑ 11.3 封锁协议
- ❑ 11.4 活锁和死锁
- ❑ 11.5 并发调度的可串行性
- ❑ 11.6 两段锁协议
- ❑ 11.7 封锁的粒度
- ❑ *11.8 其他并发控制机制
- ❑ 11.9 小结

11.9 小结

- ❑ 数据库的并发控制以事务为单位
- ❑ 数据库的并发控制通常使用封锁机制
 - 基本封锁：排它锁，共享锁
 - 多粒度封锁：排它锁，共享锁，意向共享锁，意向排它锁，共享意向排它锁
 - 三阶段封锁协议，多粒度封锁协议
- ❑ 活锁和死锁
 - 活锁的避免方法：先来先服务
 - 死锁的预防：一次封锁法，顺序封锁法
 - 死锁的检测：超时法，等待图法

- ❑ 并发事务调度的正确性
 - 串行调度
 - 可串行性（可串行化调度）
 - 并发操作的正确性则通常由两段锁协议来保证。
 - 两段锁协议是可串行化调度的充分条件，但不是必要条件
 - 冲突可串行性（冲突可串行化调度）
 - 冲突操作
 - 两段锁协议